INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
DEPARTMENT OF MECHATRONICS ENGINEERING

MECHATRONIC SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2026/2027

EXPERIMENT 3: DESIGNING A MOTION-ACTIVATED RFID SYSTEM

SECTION: 1 | GROUP: 11
SUBMISSION DATE: 25/10/2026

NO.	MATRIC NO.	NAME
1	2414273	DANISH IMAN BIN AJO
2	2414585	ALIFF EMIR BIN AHMAD EZAD
3	2416879	AHMAD AMIRULAQIL BIN ZAINORKHAIR
4	2418895	MUHAMMAD IRFAN BIN ROSHDAN

ABSTRACT

This experiment focuses on integrating multiple input and output devices with an Arduino Uno through serial communication. The system combines an MPU6050 motion sensor, an RFID module (MFRC522), a servo motor, and LED indicators to form a motion-activated access system. The experiment is divided into three main tasks: real-time motion tracking using the MPU6050, RFID-based user authentication, and full system integration that combines motion and RFID verification to control a servo motor. Python is used to visualize motion data and communicate with Arduino for access validation. Results show successful detection of circular motion and RFID authentication, enabling servo unlocking with LED feedback. This experiment demonstrates the effectiveness of serial communication in real-time sensor-actuator integration for smart control systems.

TABLE OF CONTENTS

ABSTRACT 2
TABLE OF CONTENTS 2
PART 1 2
- **1.0 INTRODUCTION & THEORY** 2
- **1.1 MATERIALS AND EQUIPMENT** 2
- **1.2 EXPERIMENTAL SETUP** 2
- **1.3 METHODOLOGY & PROGRAMMING LOGIC** 2
- **1.4 DATA COLLECTION & ANALYSIS** 2
- **1.5 RESULTS & DISCUSSION** 3
- **1.6 CONCLUSION** 3

PART 2 3
- **2.0 INTRODUCTION & THEORY** 3
- **2.1 MATERIALS AND EQUIPMENT** 3
- **2.2 EXPERIMENTAL SETUP** 3
- **2.3 METHODOLOGY & PROGRAMMING LOGIC** 3
- **2.4 DATA COLLECTION & ANALYSIS** 3
- **2.5 RESULTS & DISCUSSION** 3
- **2.6 CONCLUSION** 3

RECOMMENDATIONS 3
REFERENCES 3
ACKNOWLEDGEMENT 3
STUDENT DECLARATION 4

PART 1

1.0 INTRODUCTION & THEORY

Arduino UNO is a microcontroller used for a variety of applications in the mechatronics world. The Arduino UNO is easily programmable and usable with many electrical and electronic components which allows for projects and systems to be built quickly. In this experiment, components that were used are the MPU6050 Inertial Measurement Unit (IMU) sensor, connecting jumper wires, and a computer running a Python script which accomplishes real-time hand motion tracking with dynamic path visualization. The MPU6050 sensor is wired with its VCC and GND pins connected to the Arduino's 5V and GND headers, while its SDA and SCL pins are connected to the corresponding I2C communication pins (A4 and A5 on the Uno). The Arduino reads the 6-axis motion data—consisting of 3-axis acceleration and 3-axis angular velocity—from the sensor via the I2C bus and streams the raw values over serial communication to the computer. The Python program receives this continuous serial stream, processes the kinematic measurements, and plots the real-time spatial trajectory on an interactive graph. By using these hardware and software components together, we can accomplish real-time tracking of hand motion and display the resulting path dynamically.

1.1 MATERIALS AND EQUIPMENT

- Arduino board
- MPU6050 sensor
- Jumper wires/breadboard
- USB cable
- Computer with Arduino IDE and Python installed

1.2 EXPERIMENTAL SETUP

Pin Connections from MPU6050 to Arduino UNO:

1. VCC -> 5V
2. GND -> GND
3. SCI -> A5
4. SDA -> A4

1.3 METHODOLOGY & PROGRAMMING LOGIC

Experiment Steps:

1. Connect the Arduino UNO to the computer via USB
2. Upload the provided Arduino sketch code to the Arduino UNO
3. Observe the Serial Monitor for MPU6050 readings where it will display x-axis, y-axis and z-axis locations of the MPU6050

Task Steps:

1. Connect the Arduino UNO to the computer via USB

   Ensure that the Arduino IDE Serial Monitor is not opened before proceeding.

2. Upload the provided Python code to the Arduino UNO

   Ensure that the '*Show plots in tool window*' checkbox is unchecked in *Settings* -> *Python* -> *Plots*.

3. The plot of x-axis location vs. y-axis location of the MPU6050 is displayed in another window separate from the Python IDE.
4. The MPU6050 is picked up and spun around in a circular motion and the gesture is drawn in the plot.

Source Code Blocks:

Arduino Code Block:

Experiment:

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

void setup() {
      Serial.begin(9600);
      Wire.begin();
      mpu.initialize();
      if (!mpu.testConnection()) {
            Serial.println("MPU6050 connection failed!");
            while (1);
      }
}

void loop() {
      int ax, ay, az, gx, gy, gz;
      mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
      Serial.print(ax); Serial.print(",");
      Serial.print(ay); Serial.print(",");
      Serial.println(az);
      delay(100);
}
```

Python Code Block:

Experiment:

```
import serial
import matplotlib.pyplot as plt

ser = serial.Serial('COM4', 9600, timeout=1) # Change to
match your system

x_vals, y_vals = [], []
plt.ion()
fig, ax = plt.subplots()
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)

line, = ax.plot([], [], 'ro')
while True:
     try:
          data = ser.readline().decode().strip()
          if data:
               values = data.split(',')
               if len(values) == 6:
                    ax_val = int(values[0])
                    ay_val = int(values[1])
                    x_vals.append(ax_val)
                    y_vals.append(ay_val)
                    line.set_data(x_vals, y_vals)
                    plt.draw()
                    plt.pause(0.05)
     except KeyboardInterrupt:
          break
```

Task:

```
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import numpy as np
import time
from collections import deque

# Change to match your system
ser = serial.Serial('COM8', 9600, timeout=0.01)
time.sleep(2)
```

```
# Use deques for efficient removal of old points
timestamps = deque()
x_vals = deque()
y_vals = deque()

scale = 1000 * 20

fig, ax = plt.subplots()
ax.set_title("X-Axis vs Y-Axis")
ax.set_xlim(-scale / 2, scale + 5000)
ax.set_ylim(-scale / 2, scale + 5000)
line, = ax.plot([], [], color='red')

def check_circular_gesture(x_arr, y_arr):
    if len(x_arr) < 16:
        return False, 0, 0

    x = np.array(x_arr)
    y = np.array(y_arr)

    cx, cy = np.mean(x), np.mean(y)

    span_x = np.max(x) - np.min(x)
    span_y = np.max(y) - np.min(y)

    if span_x < scale or span_y < scale:
        return False, cx, cy

    dx = x - cx
    dy = y - cy
    q1 = np.any((dx > 0) & (dy > 0))
    q2 = np.any((dx < 0) & (dy > 0))
    q3 = np.any((dx < 0) & (dy < 0))
    q4 = np.any((dx > 0) & (dy < 0))

    if not (q1 and q2 and q3 and q4):
        return False, cx, cy

    radii = np.sqrt(dx**2 + dy**2)
    mean_radius = np.mean(radii)
    std_radius = np.std(radii)

    is_circle = (std_radius / mean_radius) < 0.45
    return bool(is_circle), cx, cy
```

```
def update_plot(frame):
    # 1. Read all waiting data in the buffer
    while ser.in_waiting > 0:
        try:
            data = ser.readline().decode().strip()
            if data:
                values = data.split(',')
                if len(values) == 3:
                    ax_val = int(values[0])
                    ay_val = int(values[1])

                    # Store the data alongside the current
exact time
                    timestamps.append(time.time())
                    x_vals.append(ax_val)
                    y_vals.append(ay_val)
        except Exception:
            pass

    # 2. Remove points older than 3 seconds
    current_time = time.time()
    while timestamps and (current_time - timestamps[0]) >
2.5:
        timestamps.popleft()
        x_vals.popleft()
        y_vals.popleft()

    # 3. Update line and auto-scale the axes
    if x_vals:
        line.set_data(list(x_vals), list(y_vals))
        ax.relim()

    detected, cx, cy = check_circular_gesture(x_vals,
y_vals)
    # print(detected)
    if detected:
        print("Circular gesture detected")
    else:
        print("Circular gesture not detected")

    return line,
```

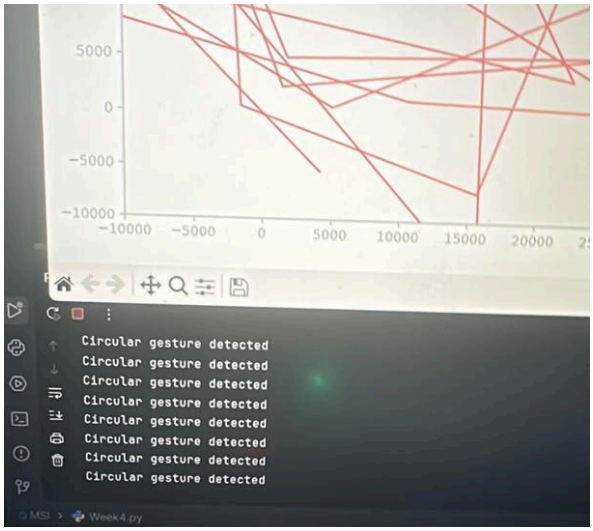
```
ani = animation.FuncAnimation(fig, update_plot,
interval=10, cache_frame_data=False)

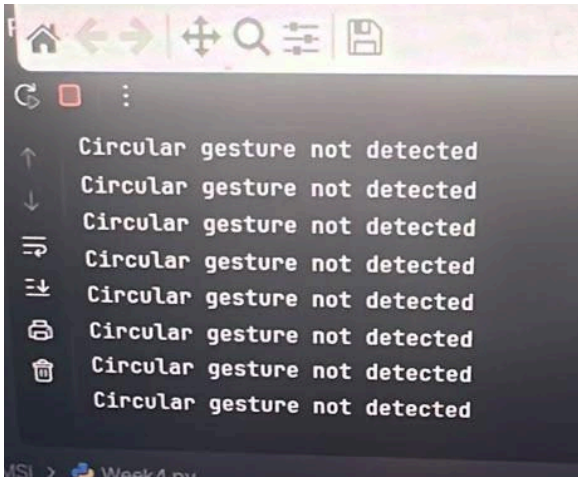
plt.show(block=True)
```

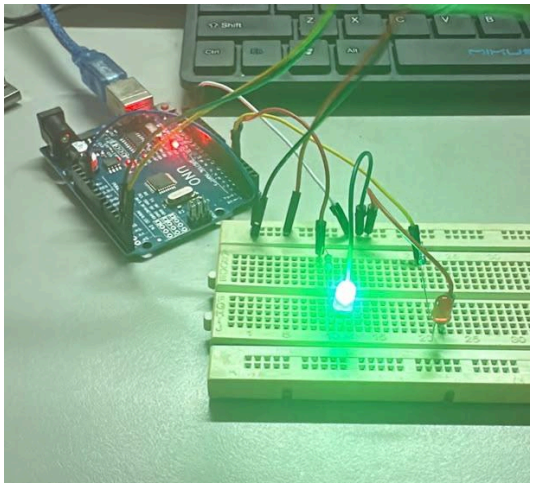
1.4 DATA COLLECTION & ANALYSIS

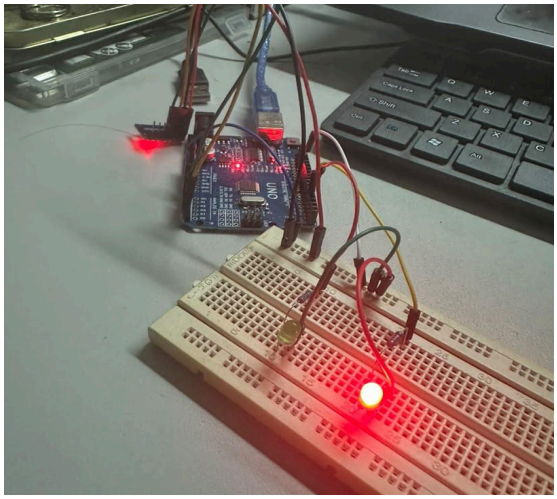
Condition	**Response**
Circular gesture is performed	Green LED turns on
Circular gesture is not performed	Red LED turns on

1.5 RESULTS & DISCUSSION









One error we encountered was an issue with *matplotlib.pyplot* where the external window that should have displayed the MPU6050 motion was stuck in an infinite loading loop. Our solution was that we resorted to using *matplotlib.animation* to display the graphical representation in a sequence of images that replicated the look of a real-time graphical representation.

1.6 CONCLUSION

The objective of capturing and visualizing real-time motion data from the MPU6050 sensor was achieved. The sensor successfully tracked hand movement, and circular gestures were detected using Python visualization. This experiment strengthened understanding of I2C communication and serial data handling between Arduino and Python.

PART 2

2.0 INTRODUCTION & THEORY

Arduino UNO is a microcontroller used for a variety of applications in the mechatronics world. The Arduino UNO is easily programmable and usable with many electrical and electronic components which allows for projects and systems to be built quickly. In this experiment, components that were used are an RC522 RFID reader module, an MPU6050 motion sensor, an SG90 servo motor, indicator LEDs with current-limiting resistors, and RFID tags which accomplishes an integrated two-factor smart access system. The RFID module is interfaced with the Arduino using SPI communication pins (such as D10 to D13) to read the unique identification (UID) of a presented card, while the MPU6050 is wired to the I2C lines (A4 and A5) to capture dynamic movement. The servo motor and indicator LEDs are connected to separate digital pins to act as physical actuators indicating access state. During operation, the Arduino first reads the scanned tag's UID to verify authorization. Upon successful credential verification, the system prompts the user to perform a circular gesture, reading and analyzing the inertial measurements from the MPU6050 to validate the motion. Based on whether the motion criteria are satisfied, the microcontroller sends an access confirmation ('A') or denial ('D') signal, concurrently driving the servo motor to unlock or remain locked while updating the status LEDs accordingly. By using these components together, we can accomplish an integrated, multi-stage smart access control mechanism.

2.1 MATERIALS AND EQUIPMENT

- Arduino board
- RFID reader (MFRC522) + RFID tags/cards
- Servo motor
- Red & Green LEDs + resistors
- MPU6050 sensor (from Part 1)
- Jumper wires and breadboard

2.2 EXPERIMENTAL SETUP

1. VCC -> 5V
2. GND -> GND
3. SCI -> A5
4. SDA -> A4
5. Servo -> 9
6. RFID -> computer port
7. LEDs -> 3 , 4

2.3 METHODOLOGY & PROGRAMMING LOGIC

Experiment Steps:

1. Connect rfid to the computer
2. Connect servo and MPU6050 and LED to arduino uno

3. Upload code to the arduino
4. Touch the card to RFID
5. Perform the circular motion
6. Observe the led whether turn green or red
7. Record

Task Steps:

1. Connect the Arduino UNO to the computer via USB

   Ensure that the Arduino IDE Serial Monitor is not opened before proceeding.

2. Upload the provided Python code to the Arduino UNO

   Ensure that the '*Show plots in tool window*' checkbox is unchecked in *Settings -> Python Plots*.

3. Touch the RFID card to the card reader (ensure that it will not work with unregistered card)
4. The terminal will ask to perform a circular motion using the MPU6050 (ensure that it will detect motion, no need for perfect circular motion)
5. The plot of x-axis location vs. y-axis location of the MPU6050 is displayed in another window separate from the Python IDE.
6. The MPU6050 is picked up and spun around in a circular motion and the gesture is drawn in the plot.
7. The servo will be turning when the circular motion is detected and verified
8. The light will light green when the servo turn indicating the user passed the verification or will light red if didn't pass the verification
9. The LED will turn off the servo and will turn to its original place and the user will be prompted to tap the card again.

Source Code Blocks:

Arduino Code Block:

Experiment:

```
#include <Servo.h>
Servo servo;
int servoPosition = 90;
void setup() {
      servo.attach(9);
      servo.write(servoPosition);
      pinMode(4, OUTPUT); // Green LED
      pinMode(3, OUTPUT); // Red LED
      Serial.begin(9600);
```

```
}

void loop() {
     if (Serial.available() > 0) {
          char command = Serial.read();
          if (command == 'A') {
               servo.write(180); // Unlock
               digitalWrite(4, HIGH);
               digitalWrite(3, LOW);
          } else if (command == 'D') {
               servo.write(90); // Lock
               digitalWrite(4, LOW);
               digitalWrite(3, HIGH);
               }
     }
}

#include <SPI.h>
#include <MFRC522.h>
#define SS_PIN 10
#define RST_PIN 8

MFRC522 rfid(SS_PIN, RST_PIN);
void setup() {
     Serial.begin(9600);
     SPI.begin();
     rfid.PCD_Init();

}

void loop() {
if (!rfid.PICC_IsNewCardPresent()) return;
if (!rfid.PICC_ReadCardSerial()) return;

Serial.print("UID:");
for (byte i = 0; i < rfid.uid.size; i++) {
          Serial.print(rfid.uid.uidByte[i], HEX);
}
Serial.println();
rfid.PICC_HaltA();
}
```

Python Code Block:

Experiment:

```
import serial
import time
SERIAL_PORT = 'COM4'
BAUD_RATE = 9600
authorized_cards = ["04AABBCCDD", "1234567890"]
def detect_circular_motion():
# To do: implement real motion detection
# You should reuse or enhance your motin tracking logic
from Part 1
print("Perform circular motion now...")
time.sleep(3)
# Return True if pattern is circular
return True

def main():
      ser = serial.Serial(SERIAL_PORT, BAUD_RATE)
      try:
      while True:
            line = ser.readline().decode().strip()
            if line.startswith("UID:"):
                  uid = line[4:].strip().upper()
                  if uid in authorized_cards:
                        print("RFID authorized. Perform
                  motion...")
                        if detect_circular_motion():
                              print("Motion verified. Access
                        granted.")
                              ser.write(b'A')
                        else:
                              print("Motion invalid. Access
                        denied.")
                              ser.write(b'D')
                  else:
                        print("UID unauthorized. Access
                  denied.")
                        ser.write(b'D')
                  time.sleep(0.1)
      except KeyboardInterrupt:
            print("Program terminated.")
      finally:
            ser.close()

if __name__ == "__main__":
```

```
    main()
```

Task:

```
from collections import deque
import time
import matplotlib.pyplot as plt
import numpy as np
import serial

# --- Configuration ---
SERIAL_PORT = 'COM10'
BAUD_RATE = 9600
AUTHORIZED_CARDS = ['0013084287']
SCALE = 1000 * 20
GESTURE_TIMEOUT = 10.0  # Seconds allowed to perform
circle

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=0.1)

def check_circular_gesture(x_arr, y_arr):
  """Evaluates whether the collected coordinates form a
complete circular pattern."""
  if len(x_arr) < 16:
    return False, 0, 0

  x = np.array(x_arr)
  y = np.array(y_arr)

  cx, cy = np.mean(x), np.mean(y)

  span_x = np.max(x) - np.min(x)
  span_y = np.max(y) - np.min(y)

  # Check minimum amplitude threshold
  if span_x < SCALE or span_y < SCALE:
    return False, cx, cy

  # Check quadrant traversal
  dx = x - cx
  dy = y - cy
  q1 = np.any((dx > 0) & (dy > 0))
  q2 = np.any((dx < 0) & (dy > 0))
  q3 = np.any((dx < 0) & (dy < 0))
```

```
  q4 = np.any((dx > 0) & (dy < 0))

  if not (q1 and q2 and q3 and q4):
    return False, cx, cy

  # Check radial variance
  radii = np.sqrt(dx**2 + dy**2)
  mean_radius = np.mean(radii)
  std_radius = np.std(radii)

  is_circle = (std_radius / mean_radius) < 0.45
  return bool(is_circle), cx, cy


def detect_circular_motion(ser, duration=GESTURE_TIMEOUT):
  """Opens a live tracking window, reads MPU6050 data from
serial,

  plots the path, and verifies if a circle was completed.
  """
  print(f"\n[MOTION] Perform circular motion now
({duration}s window)...")

  # Flush stale accelerometer readings from the buffer
  ser.reset_input_buffer()

  timestamps = deque()
  x_vals = deque()
  y_vals = deque()

  # Set up live plot
  plt.ion()
  fig, ax = plt.subplots(figsize=(5, 5))
  ax.set_title("Perform Circular Gesture Now")
  ax.set_xlim(-SCALE / 2, SCALE + 5000)
  ax.set_ylim(-SCALE / 2, SCALE + 5000)
  (line,) = ax.plot([], [], color='red', lw=2)
  plt.show()

  start_time = time.time()
  gesture_passed = False

  while (time.time() - start_time) < duration:
    # 1. Read all waiting MPU6050 data packets
    while ser.in_waiting > 0:
```

```
      try:
        raw_line = ser.readline().decode('utf-8',
errors='ignore').strip()
        if raw_line:
          parts = raw_line.split(',')
          if len(parts) >= 2:
            ax_val = int(parts[0])
            ay_val = int(parts[1])

            timestamps.append(time.time())
            x_vals.append(ax_val)
            y_vals.append(ay_val)
      except (ValueError, UnicodeDecodeError):
        pass

    # 2. Retain points within the rolling 2.5-second
gesture window
    now = time.time()
    while timestamps and (now - timestamps[0]) > 1.5:
      timestamps.popleft()
      x_vals.popleft()
      y_vals.popleft()

    # 3. Update trajectory plot
    if x_vals:
      line.set_data(list(x_vals), list(y_vals))
      ax.relim()
      fig.canvas.flush_events()

    # 4. Check circle geometry
    is_circle, _, _ = check_circular_gesture(x_vals,
y_vals)
    if is_circle:
      print(">> Circular gesture verified!")
      gesture_passed = True
      break

    plt.pause(0.01)

  plt.close(fig)
  return gesture_passed


def main():
  try:
```

```
    time.sleep(2)  # Wait for Arduino bootloader to settle
    print(f"Connected to Arduino on {SERIAL_PORT}")
  except serial.SerialException as e:
    print(f"Serial Connection Error: {e}")
    return

  print("\n================ SYSTEM READY
================")
  print("Tap your RFID card on the USB Dongle...")

  try:
    while True:
      # Block until the USB Dongle types the card UID and
sends Enter
      scanned_uid = input("\n[RFID] Tap Card:
").strip().upper()
      if not scanned_uid:
        continue

      print(f"Scanned UID: {scanned_uid}")

      if scanned_uid in AUTHORIZED_CARDS:
        print("RFID Authorized. Prepare for gesture...")
        time.sleep(1)

        if detect_circular_motion(ser):
          print(">> Access Granted: Green LED ON, Servo
Unlocking.")
          ser.write(b'A')
        else:
          print(">> Motion Verification Failed: Red LED
ON.")
          ser.write(b'D')
      else:
        print(">> Unauthorized UID: Access Denied. Red LED
ON.")
        ser.write(b'D')

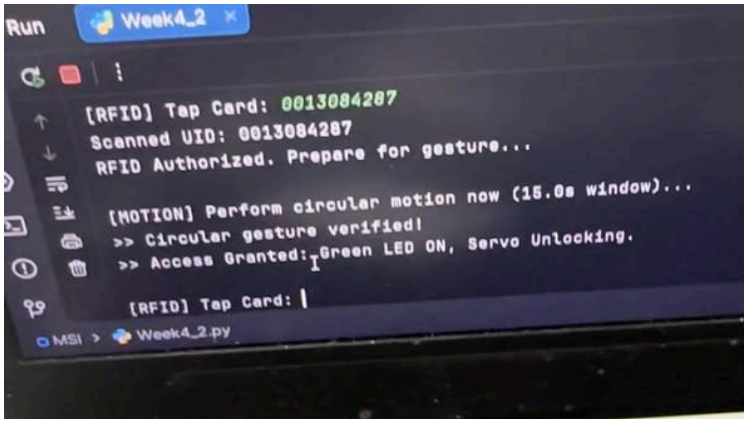
  except KeyboardInterrupt:
    print("\nProgram terminated by user.")
  finally:
    if ser.is_open:
      ser.close()
```

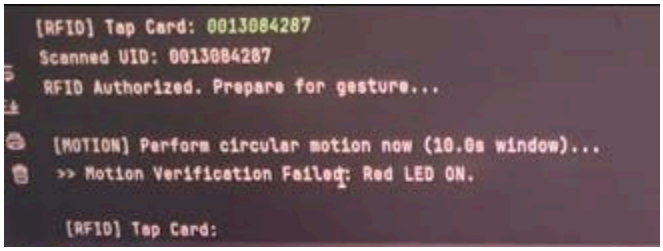
```
if __name__ == '__main__':
    main()
```

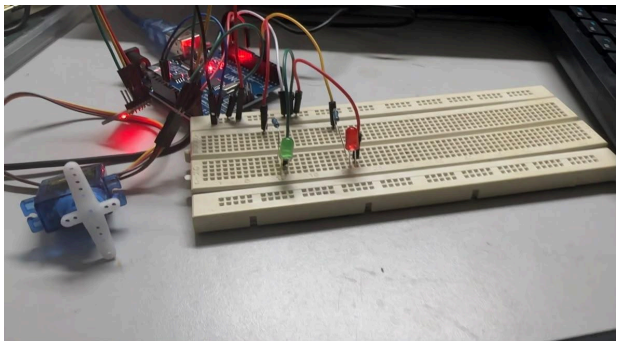
2.4 DATA COLLECTION & ANALYSIS

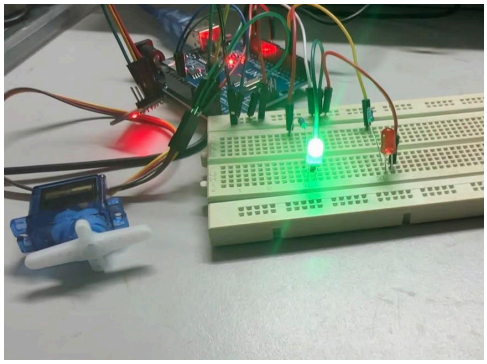
Condition	Response
Unauthorize RFID card reading	The LED will light red and the user will be ask to tap registered card
Authorize card reading	The LED will light green and the user will be ask to perform circular motion using the MPU6050
Not detect circular motion	The LED will light red and the user need to tap the card again and perform the motion again
Detect circular motion	The LED will light green and the servo will turn and a delay for 3 second before the servo will turn to its initial position and the LED will turn off

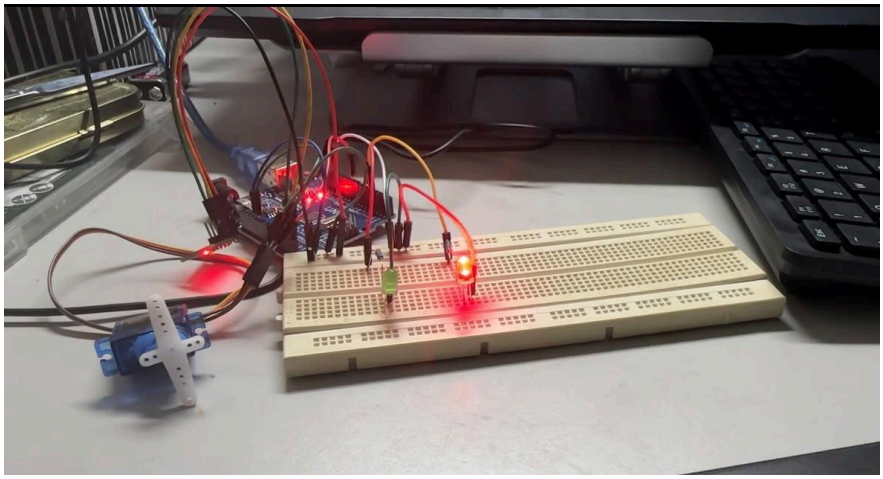
2.5 RESULTS & DISCUSSION











The system successfully demonstrated two-level verification using RFID authentication and IMU-based motion detection. The interaction between Python and Arduino through serial communication was reliable and responsive.

Overall, the system achieved its objective which is to only activate the servo and green LED after a correct RFID scan and a valid motion gesture. The responses from both hardware (LEDs, servo) and software (Python console messages) confirmed full integration and accurate functionality.

Furthermore, the experiment demonstrated how serial communication can be used to interface sensors with a computer for real-time visualization and control. The MPU6050 proved to be a reliable motion sensor, though sensitive to external vibrations and orientation changes. While Python provided a flexible platform for interfacing with the hardware, the detection algorithm could be improved by adding more advanced numerical pattern recognition. Ultimately, this experiment successfully integrates motion-based verification with RFID for a complete smart access system.

2.6 CONCLUSION

The objective of capturing and visualizing real-time motion data from the MPU6050 sensor was achieved. The sensor successfully tracked hand movement, and circular gestures were detected

using Python visualization. This experiment strengthened understanding of I2C communication and serial data handling between Arduino and Python.

RECOMMENDATIONS

1. Implement moving average filters to reduce noise.

2. Include gyroscope data for more accurate motion pattern detection.

3. Optimize Python script to auto-detect circular motion algorithmically.

4. Use higher baud rate for smoother live plotting.

5. Calibrate the sensor for consistent readings.

REFERENCES

[1] LME Editorial Staff, "What is RFID? How It Works? Interface RC522 RFID Module with Arduino," *Last Minute Engineers*, Apr. 7, 2025. [Online]. Available: https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/

[2] R. Santos, "MFRC522 RFID Reader with Arduino Tutorial," *Random Nerd Tutorials*, Apr. 23, 2023. [Online]. Available: https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

[3] S. Santos, "Arduino Time Attendance System with RFID," *Random Nerd Tutorials*, Apr. 2, 2019. [Online]. Available: https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/

[4] Instructables, "Arduino + MFRC522 RFID READER," *Instructables*, Sep. 22, 2017. [Online]. Available: https://www.instructables.com/Arduino-MFRC522-RFID-READER/

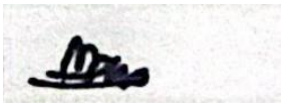
ACKNOWLEDGEMENT

Alhamdulillah, all praise and gratitude be to Allah SWT for His blessings, guidance, and mercy that enabled us to successfully complete this project entitled DESIGNING A MOTION-ACTIVATED RFID SYSTEM. Throughout the development process, we faced several challenges, but with His will and guidance, we managed to overcome them and complete the project successfully. We would also like to express our heartfelt appreciation to our lecturer for their continuous guidance, support, and valuable feedback throughout this project. Their advice has greatly helped us in understanding both the theoretical and practical aspects of microcontroller systems and serial communication.

STUDENT DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgments, and that the original work contained herein has not been undertaken or done by unauthorized sources or persons.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: DANISH IMAN BIN AJO Matric No: 2414273	Read [/] Understand [/] Agree [/]
	Name: ALIFF EMIR BIN AHMAD EZAD Matric No: 2414585	Read [/] Understand [/] Agree [/]
	Name:AHMAD AMIRULAQIL BIN ZAINORKHAIR Matric No: 2416879	Read [/] Understand [/] Agree [/]
	Name: MUHAMMAD IRFAN BIN ROSHDAN Matric No: 2418895	Read [/] Understand [/] Agree [/]